

API de Foco e Produtividade

Guia de Arquitetura — Conversa Completa

Este documento registra a conversa entre o desenvolvedor e o arquiteto de software sobre o desafio técnico de construção de um *Log de Performance*, incluindo a arquitetura sugerida, estrutura de pastas, decisões técnicas e checklist de entrega.

Prazo	Próxima segunda-feira
Linguagem	Python 3.x
Framework	FastAPI (recomendado)
Armazenamento	In-memory ou SQLite
Endpoints	POST /registro-foco · GET /diagnostico-produtividade

1. O Desafio

O objetivo é criar o backend de um **Log de Performance** — uma API que ajuda desenvolvedores e estudantes a registrar seu nível de foco durante sessões de trabalho e, ao final, entregue um diagnóstico inteligente do período.

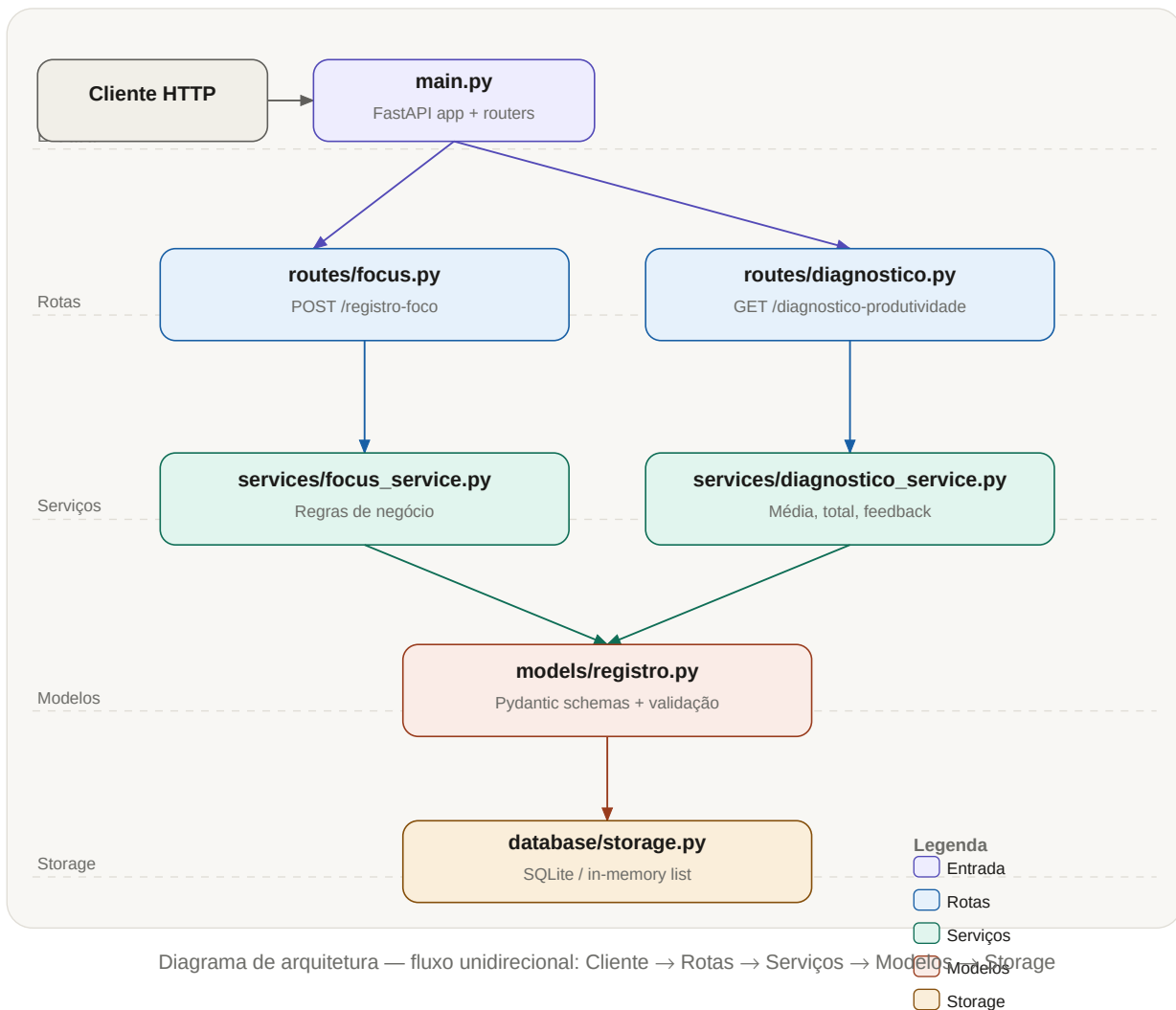
A proposta surgiu da observação de que muitas vezes trabalhamos muito mas produzimos pouco. A API permite que o usuário registre blocos de trabalho com nível de foco (1–5), duração e um comentário descritivo.

Regras de Entrega

Prazo: próxima segunda-feira. Uso de IA é permitido — mas os artefatos gerados devem ser commitados. Faça fork do repositório e avise o avaliador (wouerner) no LinkedIn ao terminar.

2. Arquitetura da Solução

A arquitetura segue o padrão de **camadas (Layered Architecture)**: simples o suficiente para um desenvolvedor júnior executar, mas organizado da forma que um sênior esperaria encontrar. Cada camada tem responsabilidade única e bem definida.



3. Estrutura de Pastas

Cada diretório tem uma responsabilidade clara. Nunca misture lógica de negócio com código HTTP, e nunca acesse o banco de dados diretamente de uma rota.

```

focus-log-api/
├── app/
│   ├── __init__.py
│   └── main.py # Ponto de entrada – FastAPI app + registro de routers
├── routes/ # Camada HTTP: recebe e responde requisições
│   ├── __init__.py
│   ├── focus.py # POST /registro-foco
│   └── diagnostico.py # GET /diagnostico-produtividade
├── services/ # Lógica de negócio: cálculos, regras, feedback
│   ├── __init__.py
│   ├── focus_service.py # Valida e salva um registro de foco
│   └── diagnostico_service.py # Calcula média, total e gera mensagem de feedback
├── models/
│   └── registro.py
└── database/
    └── storage.py
  
```

```

■ ■
■ ■■■ models/                # Schemas Pydantic (contratos de dados)
■ ■ ■■■ __init__.py
■ ■ ■■■ registro.py          # RegistroFocoInput, RegistroFocoOutput, DiagnosticoOutput
■ ■
■ ■■■ database/              # Persistência: lista in-memory ou SQLite
■ ■■■ __init__.py
■ ■■■ storage.py
■
■■■ tests/
■ ■■■ __init__.py
■ ■■■ test_focus.py
■ ■■■ test_diagnostico.py
■
■■■ .gitignore
■■■ requirements.txt
■■■ README.md

```

4. Responsabilidade de Cada Camada

routes/ — Camada HTTP

routes/

Recebe a requisição do cliente, delega ao serviço e retorna a resposta. Nunca contém lógica de negócio — apenas orquestra a chamada.

services/ — Camada de Negócio

services/

Contém toda a lógica: cálculos de média, geração de mensagens de feedback, validações de regra. Não conhece nada sobre HTTP.

models/ — Schemas de Dados

models/

Define a 'forma' dos dados com Pydantic. Separe modelos de entrada (Input) dos de saída (Output). A validação é automática.

database/ — Persistência

database/

Centraliza todo acesso a dados. Para simplicidade: lista global. Para diferencial: SQLite com a biblioteca padrão sqlite3.

Regra de ouro da arquitetura

A dependência flui em uma só direção: routes → services → database. Nenhuma camada chama a camada acima dela. Isso mantém o código testável e organizado.

5. Modelos de Dados Sugeridos

Defina os modelos **antes** de escrever qualquer rota. O Pydantic cuida da validação automaticamente — menos código, menos bug.

```
# app/models/registro.py

from pydantic import BaseModel, Field
from typing import Optional
from datetime import datetime
from enum import Enum

class CategoriaEnum(str, Enum):
    coding = "coding"
    reuniao = "reunião"
    estudo = "estudo"
    outro = "outro"

class RegistroFocoInput(BaseModel):
    nivel_foco: int = Field(..., ge=1, le=5,
                           description="1=distraído, 5=flow")
    tempo_minutos: int = Field(..., gt=0,
                               description="Duração da sessão em minutos")
    comentario: str = Field(..., min_length=1)
    # Campos extras (diferencial)
    categoria: Optional[CategoriaEnum] = None
    tags: Optional[list[str]] = []

class RegistroFocoOutput(RegistroFocoInput):
    id: int
    criado_em: datetime

class DiagnosticoOutput(BaseModel):
    total_registros: int
    media_foco: float
    tempo_total_minutos: int
    mensagem_feedback: str
    distribuicao_foco: dict # {"1": 2, "2": 0, ...} - diferencial
```

6. Lógica de Feedback (Diferencial)

A mensagem de feedback é gerada automaticamente no serviço de diagnóstico, baseada na média do nível de foco de todos os registros.

```
# app/services/diagnostico_service.py

def gerar_feedback(media: float, total_minutos: int) -> str:
    if media < 2:
        return ("Muitas distrações hoje. "
                "Tente blocos menores com o método Pomodoro.")
    elif media < 3:
        return ("Foco abaixo do ideal. "
                "Pausas mais longas e menos notificações podem ajudar.")
    elif media < 4:
        return "Bom desempenho! Você está no caminho certo."
    elif media < 5:
        return ("Ótimo foco! Continue assim e "
                "proteja seus blocos de concentração.")
    else:
        return ("Estado de flow! Você está em uma "
                "maratona produtiva de alto nível!")
```

7. Tratamento de Erros

O FastAPI + Pydantic tratam a maioria dos erros automaticamente. Use os campos do modelo para definir as restrições e deixe o framework trabalhar.

Situação	Status HTTP	Como tratar
nivel_foco fora de 1–5	422	Pydantic ge=1, le=5 (automático)
tempo_minutos ≤ 0	422	Pydantic gt=0 (automático)
comentario vazio	422	Pydantic min_length=1 (automático)
Sem registros no diagnóstico	200	Retorne mensagem amigável no JSON
Erro interno inesperado	500	Deixe o FastAPI tratar

8. Dependências e Como Rodar

requirements.txt

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
pydantic==2.7.1
```

Passo a passo para rodar

```
# 1. Clone e entre na pasta
git clone <seu-fork>
cd focus-log-api

# 2. Crie o ambiente virtual
python -m venv .venv
source .venv/bin/activate      # Linux/Mac
.venv\Scripts\activate        # Windows

# 3. Instale as dependências
pip install -r requirements.txt

# 4. Suba o servidor
uvicorn app.main:app --reload

# 5. Acesse a documentação interativa
# http://127.0.0.1:8000/docs
```

9. Checklist de Entrega

Revise este checklist antes de enviar o link do repositório:

Obrigatório

- Estrutura de pastas conforme o guia de arquitetura
- POST /registro-foco funcionando com validação Pydantic
- GET /diagnostico-produtividade retornando média, total e feedback
- Tratamento de erros para entradas inválidas (fora do range, campos vazios)
- README.md explicando como rodar o projeto
- .gitignore com .venv/, __pycache__/, *.db

Diferencial

- Campo categoria (coding, reunião, estudo, outro)
- Campo tags (lista de strings)
- distribuicao_foco no diagnóstico (contagem por nível)
- Testes unitários com pytest
- Artefatos de IA commitados no repositório (se usou IA)

10. Dicas do Arquiteto

Comece pelo modelo

Defina os schemas em `models/registro.py` antes de qualquer rota. Os dados são o contrato — tudo mais é implementação.

Use o /docs do FastAPI

O FastAPI gera uma UI interativa em `/docs` automaticamente. Teste todos os endpoints por lá antes de escrever testes.

Um commit por funcionalidade

Use mensagens claras: `feat: add POST /registro-foco`, `feat: add diagnostico endpoint`, `fix: validate nivel_foco range`.

Comente o porquê, não o quê

`# calcula média apenas se há registros (evita divisão por zero) é útil. # soma os valores não acrescenta nada.`

Gerado com Claude · Anthropic · 2026